

Design Document: Project 4

Author: Daniel Leone

Star ID: XXXXXXXXXX

Date: 04-10-2024

INTRODUCTION

This system is an iteration on a previous project that employed a buffer to retrieve data more efficiently from a large dataset contained within a CSV file. A program was created to retrieve data from the file, interpret the data, and then display a table to the console showing each State field's East, West, North, and South extreme points. The CSV did not need to be sorted and worked with a randomly sorted data set.

Each record of data consists of 6 fields containing specific geographical information. The original data is organized with a single lined header providing the column label, and each line of data is comma separated with the a '\n' char ending the record. Each record consists of a unique Zip Code.

	Zip Code,Place Name,State,County,Lat,Long
2	501,Holtsville,NY,Suffolk,40.8154,-73.0451
3	544,Holtsville,NY,Suffolk,40.8154,-73.0451
4	1001,Agawam,MA,Hampden,42.0702,-72.6227
5	1002,Amherst,MA,Hampshire,42.3671,-72.4646
6	1003,Amherst,MA,Hampshire,42.3919,-72.5248
7	1004,Amherst,MA,Hampshire,42.3845,-72.5132
8	1005,Barre,MA,Worcester,42.4097,-72.1084
9	1007,Belchertown,MA,Hampshire,42.2751,-72.411
10	1008,Blandford,MA,Hampden,42.1829,-72.9361
11	1009,Bondsville,MA,Hampden,42.2061,-72.3405
12	1010,Brimfield,MA,Hampden,42.1165,-72.1885
13	1011,Chester,MA,Hampden,42.2794,-72.9888
14	1012,Chesterfield,MA,Hampshire,42.3923,-72.8256
15	1013,Chicopee,MA,Hampden,42.1487,-72.6079
16	1014,Chicopee,MA,Hampden,42.1707,-72.6048
17	1020,Chicopee,MA,Hampden,42.1764,-72.5761
18	1021,Chicopee,MA,Hampden,42.1707,-72.6048
19	1022,Chicopee,MA,Hampden,42.1934,-72.5544
20	1026,Cummington,MA,Hampshire,42.4633,-72.9202
21	1027,Easthampton,MA,Hampshire,42.2668,-72.669
22	1028,East Longmeadow,MA,Hampden,42.0672,-72.5056
23	1029,East Otis,MA,Berkshire,42.1909,-73.0517
24	1030,Feeding Hills,MA,Hampden,42.0718,-72.6751
25	1031,Gilbertville,MA,Worcester,42.3322,-72.1986

This next iteration further processes the CSV to provide additional information to the data set to provide further details to the user of the data set and to allow for greater control. A file converter was created. The file converter takes in the data from the CSV and writes a new custom ZCR (Zipcode Record) file, maintaining the data integrity while further extrapolating information. The header is a fixed sizes with header fields providing higher level information. Each header field is organized: label + ':' + value + ";", with the ":" delimitating the label and value, and the ';' serving to delimitate the end of the header element, encapsulating the value and distinguishing it as a header element. The records are read into the converter, the length is calculated and then the record is written to the ZCR beginning at the offset of the header. The header is then updated with any additional information, such as record count, that should be modified once the data is written. An example ZCR file is provided.

```

1 File Structure Type: ZCR;
2 Version: 1.0;
3 Header Record Size: 450;
4 Record Size Integer Bytes: -1;
5 Size Format Type: ASCII;
6 Index File Name: us_postal_codes.idx;
7 Index File Schema: Key, Offset;
8 Record Count: 40933;
9 Fields Per Record: 6;
10 Field 1: Zip Code, Int;
11 Field 2: Place Name, String;
12 Field 3: State, String;
13 Field 4: County, String;
14 Field 5: Lat, Double;
15 Field 6: Long, Double;
16 Primary Key: Zip Code;
17 42 501,Holtsville,NY,Suffolk,40.8154,-73.0451
18 42 544,Holtsville,NY,Suffolk,40.8154,-73.0451
19 39 1001,Agawam,MA,Hampden,42.0702,-72.6227
20 42 1002,Amherst,MA,Hampshire,42.3671,-72.4646
21 42 1003,Amherst,MA,Hampshire,42.3919,-72.5248
22 42 1004,Amherst,MA,Hampshire,42.3845,-72.5132
23 40 1005,Barre,MA,Worcester,42.4097,-72.1084
24 45 1007,Belchertown,MA,Hampshire,42.2751,-72.411
25 42 1008,Blandford,MA,Hampden,42.1829,-72.9361

```

A new buffer system was created to handle this new file type more appropriately. The program from project with tested with the new system.

The next step in this iteration implemented a primary key index (in the case the key is the zipcode field value). The zipcode value of each record is stored, and the cumulative offset is calculated and stored within a generated IDX (index) file. The index can be read into an object that resides in memory to allow for much faster processing of the data, limiting the number of expensive file accesses from disk.

```

1  File Structure Type: IDX;
2  Version: 1.0;
3  Header Record Size: 450;
4  Record Size Integer Bytes: -1;
5  Size Format Type: ASCII;
6  Index File Name: us_postal_codes.idx;
7  Index File Schema: Key, Offset;
8  Record Count: 40933;
9  Fields Per Record: 2;
10 Field 1: Key, Int;
11 Field 2: Offset, Int;
12 Primary Key: Zip Code;
13 501,450
14 544,496
15 1001,542
16 1002,585
17 1003,631
18 1004,677
19 1005,723
20 1007,767
21 1008,816
22 1009,862
23 1010,909
24 1011,955
25 1012,999

```

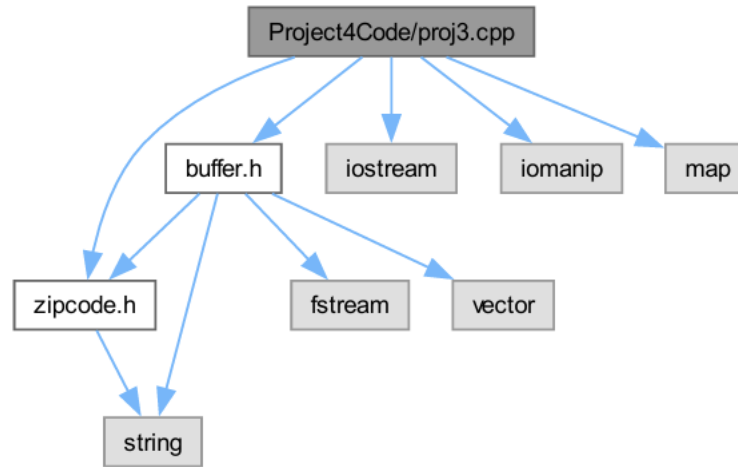
The system is designed to be executed from the command line, with the ability to accept user input for the name of the CSV or ZCR file to be processed. It provides functionality to convert a CSV file into a custom ZCR file, including a header and length indicated records. An index file (IDX) is generated and utilized to quickly locate specific records by zipcode. The record is located by user input or based on user input from the command line with the -Z flag. The record is displayed if found, or a message is displayed stating the record is not present. Multiple records can be accessed at one execution of the program.

This design document provides a detailed description of the system's design and implementation, including the purpose of the program, the classes and subprograms used, the data structures and variables, and the functions and their responsibilities. It serves as a comprehensive guide for understanding the system's design and functionality. It is intended for software developers, testers, and maintainers who are involved in the development, testing, and maintenance of the system. It can also be used by end-users who are interested in understanding the system's design and functionality. The document is organized into sections for easy navigation and understanding. Each section provides a detailed description of a specific aspect of the system's design and implementation. The document is written in clear and concise language, with diagrams and

examples where necessary to aid understanding. It is regularly updated to reflect the latest changes and improvements to the system.

SUBPROGRAMS

File Name: proj3.cpp



Purpose of Program: The program is designed to analyze geographical data from a CSV file and display extreme zip codes (easternmost, westernmost, northernmost, and southernmost) for each state.

Output: A formatted table will display for each unique value in the state field with the extremes of latitude and longitude

Executable: proj3_original

Classes: buffer.cpp, zipcode.cpp

Additional Files: us_postal_codes.csv, us_postal_codes_ROWS_RANDOMIZED.csv

Summary: It reads geographical data from a CSV file, processes it to find extreme zip codes for each state, and displays the results in a formatted table. The program reads geographical data from a CSV file using the Buffer class, processes it to find extreme zip codes for each state, and displays the results in a formatted table. The program utilizes the RecordBuffer class to store and retrieve records. It also uses the Zipcode class for handling zipcode data. It uses various helper functions for data processing and output formatting.

Data Structures/Types:

- uses std::map for storing the extreme coordinates for each state.

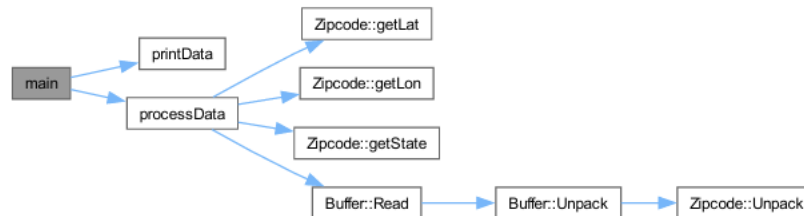
Variables:

- inputFileName (to store the name of the CSV file)
- buffer (an object of Buffer class for handling the CSV file)

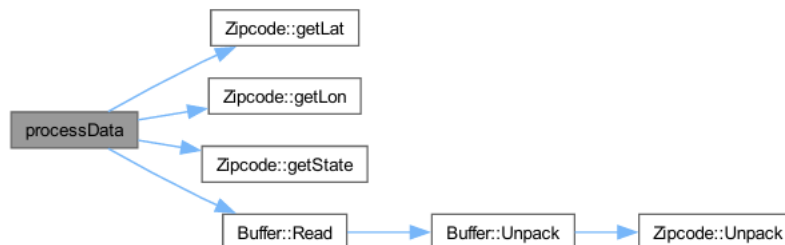
- easternmost, westernmost, northernmost, southernmost (maps to hold the extreme coordinates for each state).

Functions:

- main():
 - The main function of the program. It asks the user for the input file name, creates a Buffer object for the CSV file, creates maps to hold the extreme coordinates for each state, processes the data, and prints the data.

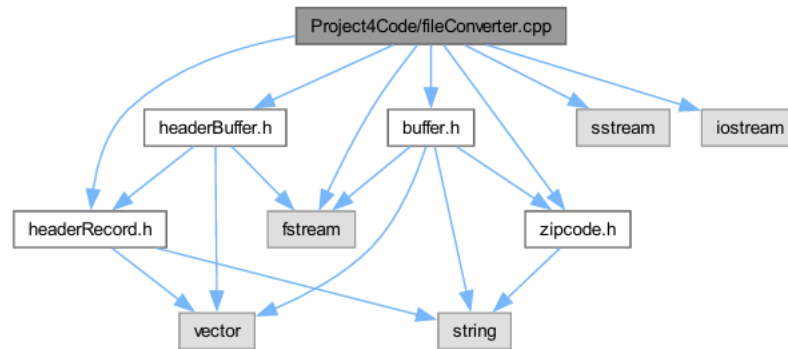


- `processData(Buffer& buffer, std::map<std::string, double>& easternmost, std::map<std::string, double>& westernmost, std::map<std::string, double>& northernmost, std::map<std::string, double>& southernmost):`
 - A function to read and process the data from the file. It reads each line from the file, gets the current state, and updates the extreme coordinates for this state.



- `printData(const std::map<std::string, double>& easternmost, const std::map<std::string, double>& westernmost, const std::map<std::string, double>& northernmost, const std::map<std::string, double>& southernmost):`
 - A function to print the extreme coordinates for each state. The implementation of this function is not provided in the given code. It is assumed that it prints the data in a formatted table.

File Name: fileConverter.cpp



Purpose of Program: The program is designed to convert a CSV file into a custom ZCR (Zipcode Record) file. It reads zipcode record data from a CSV file and converts it into a custom ZCR file that includes a header and length indicated records.

Output: A newly formatted ZCR file having the same name of the original CSV but with new extension. File will have a header and records will now be length indicated while maintaining comma delimitation.

Executable: converter

Classes: headerBuffer.cpp, headerRecord.cpp, buffer.cpp, zipcode.cpp

Additional Files: any .csv following the Zipcode formatting

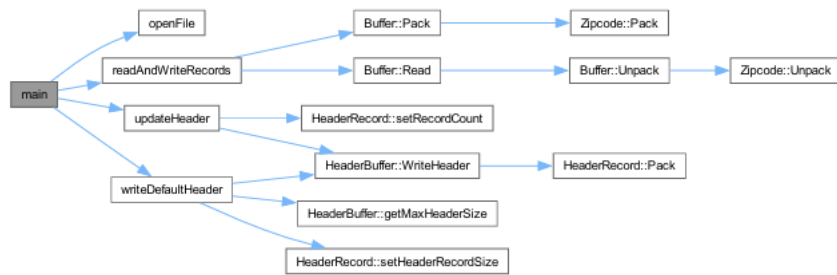
Summary: The program reads Zipcode record data from a CSV file using the Buffer class, converts it into a custom ZCR file that includes a header and length indicated records. It uses various helper functions for data processing and output formatting. The program utilizes the HeaderBuffer class to handle the header buffer and the HeaderRecord class for handling the header record. It also uses the Buffer class for handling the data buffer in and the Zipcode class for handling zipcode data.

Data Structures/Types:

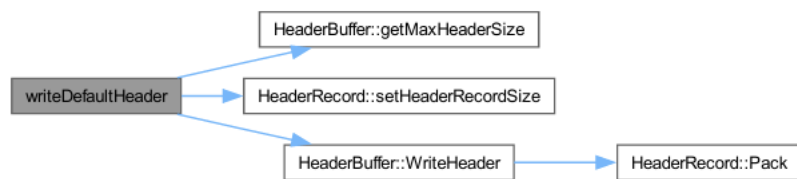
- csvFileName (to store the name of the CSV file)
- zcrFile (an object of fstream class for handling the ZCR file)
- recordCount (an integer to hold the number of records).

Functions:

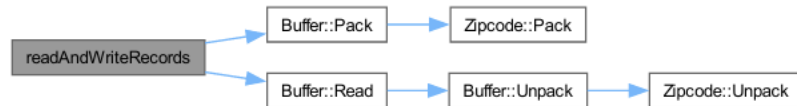
- main():
 - The main function of the program. It asks the user for the input file name, creates a Buffer object for the CSV file



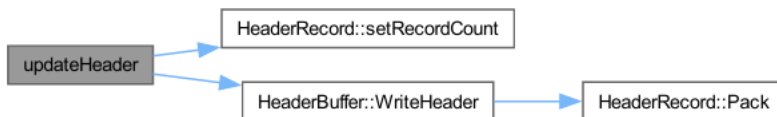
- `writeDefaultHeader(std::fstream& zcrFile):`
 - A function to write a default header to the .zcr file.



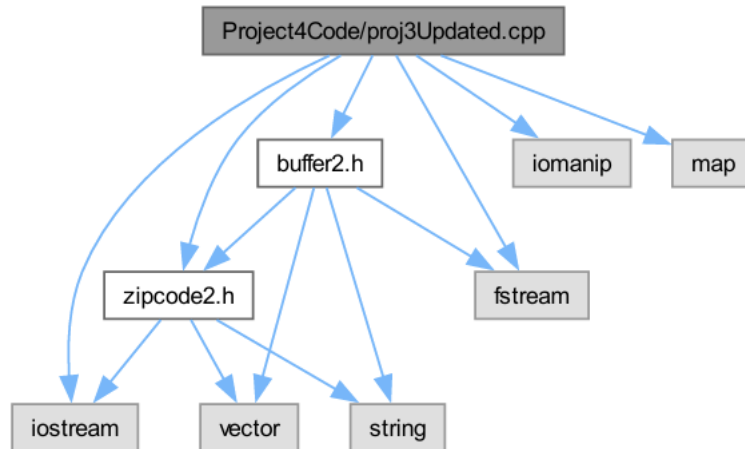
- `readAndWriteRecords(const std::string& csvFileName, std::fstream& zcrFile):`
 - A function to read the .csv and write records to .zcr.



- `updateHeader(std::fstream& zcrFile, int recordCount):`
 - A function to update the header of the .zcr file.



File Name: proj3Updated.cpp



Purpose of Program: The program is designed to analyze geographical data from a ZCR file and display extreme zip codes (easternmost, westernmost, northernmost, and southernmost) for each state.

Output: A formatted table will display for each unique value in the state field with the extremes of latitude and longitude

Executable: proj_updated

Classes: buffer2.cpp, zipcode2.cpp

Additional Files: us_postal_codes.zcr, us_postal_codes_ROWS_RANDOMIZED.zcr

Summary: It reads geographical data from a ZCR file, processes it to find extreme zip codes for each state, and displays the results in a formatted table. The program reads geographical data from a ZCR file using the Buffer2 class, processes it to find extreme zip codes for each state, and displays the results in a formatted table. The program utilizes the RecordBuffer class to store and retrieve records. It also uses the Zipcode2 class for handling zipcode data. It uses various helper functions for data processing and output formatting.

Data Structures/Types:

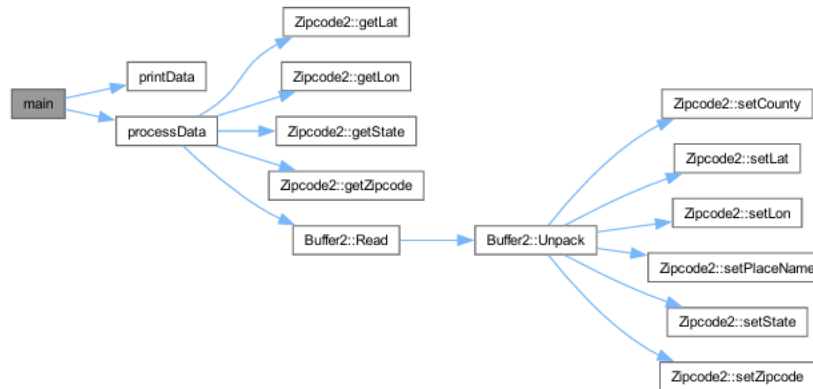
- uses std::map for storing the extreme coordinates for each state.

Variables:

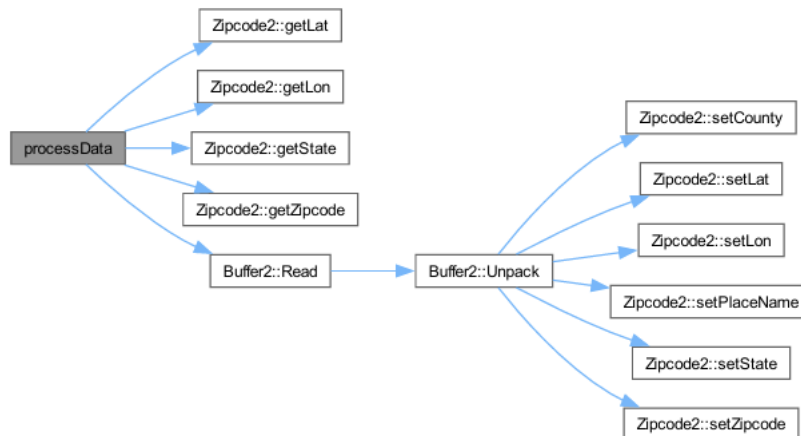
- inputFileName (to store the name of the ZCR file)
- buffer (an object of Buffer2 class for handling the ZCR file)
- easternmost, westernmost, northernmost, southernmost (maps to hold the extreme coordinates for each state).

Functions:

- **main():**
 - The main function of the program. It asks the user for the input file name, creates a Buffer2 object for the ZCR file, creates maps to hold the extreme coordinates for each state, processes the data, and prints the data.

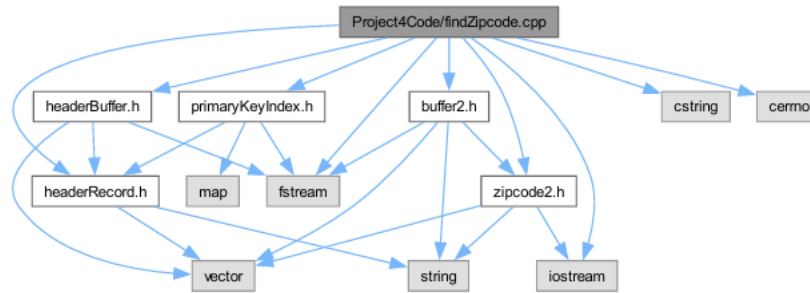


- **processData(Buffer2& buffer, std::fstream& file, std::map<std::string, double>& easternmost, std::map<std::string, double>& westernmost, std::map<std::string, double>& northernmost, std::map<std::string, double>& southernmost):**
 - A function to read and process the data from the file. It reads each line from the file, gets the current state, and updates the extreme coordinates for this state.



- **printData(const std::map<std::string, double>& easternmost, const std::map<std::string, double>& westernmost, const std::map<std::string, double>& northernmost, const std::map<std::string, double>& southernmost):**
 - A function to print the extreme coordinates for each state. The implementation of this function is not provided in the given code. It is assumed that it prints the data in a formatted table.

File Name: findZipcode.cpp



Purpose of Program: The program is designed to find and display zip codes from a ZCR file. It reads zipcode record data from a custom ZCR file that includes a header and length indicated records.

Output: Displays the requested Zipcode record if located. Multiple Zipcode records can be displayed at a time via -Z command flag.

Executable: Any of the following. Note that each executable uses a differently organized ZCR file.

- proj4_norm
- proj4_rand
- proj4_norm_place
- proj4_rand_state.

Classes: headerBuffer.cpp, headerRecord.cpp, buffer2.cpp, zipcode2.cpp, primaryKeyIndex.cpp

Additional Files:

- us_postal_codes.zcr, us_postal_codes.idx
- us_postal_codes_ROWS_RANDOMIZED.zcr, us_postal_codes_ROWS_RANDOMIZED.idx
- us_postal_codes_place.zcr, us_postal_codes_place.idx
- us_postal_codes_rr_state.zcr, us_postal_codes_rr_state.idx

Summary: The program reads zipcode record data from a custom ZCR file using the Buffer2 class, and an index file (IDX) is generated and utilized to quickly locate specific records by zipcode. The record is located based on user input or simply from the command line with the -Z flag. The record is displayed if found, or a message is displayed stating the record is not present. Multiple records can be accessed at one execution of the program. The program utilizes the HeaderBuffer class to handle the header buffer and the HeaderRecord class for handling the header record. It also uses the Buffer2 class for handling the data buffer, the Zipcode2 class for handling zipcode data, and the PrimaryKeyIndex class for handling the primary key index. It uses various helper functions for data processing and output formatting.

Data Structures/Types:

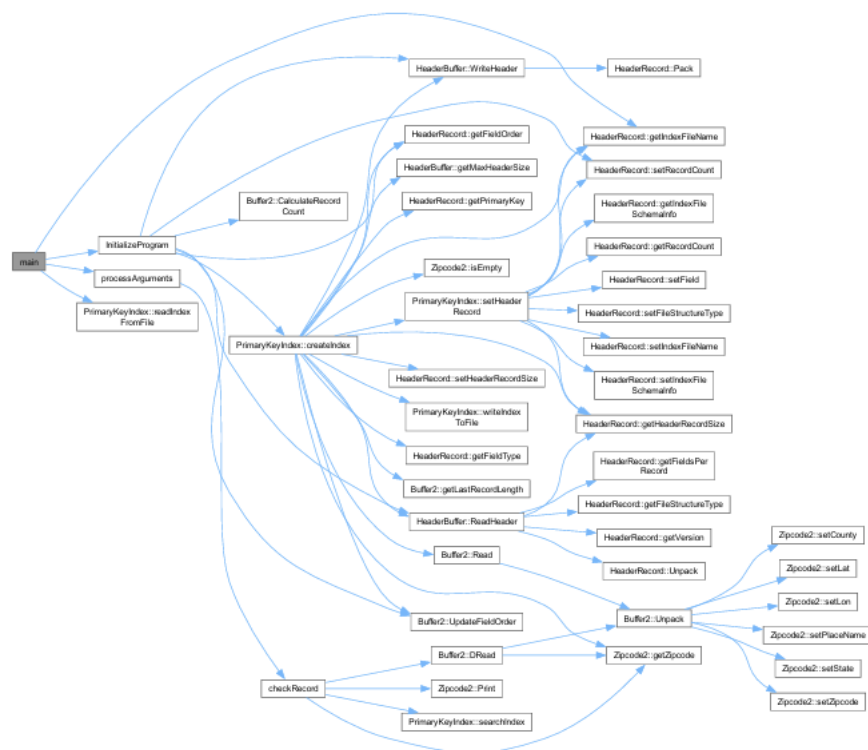
- std::map for storing the extreme coordinates for each state.

Variables:

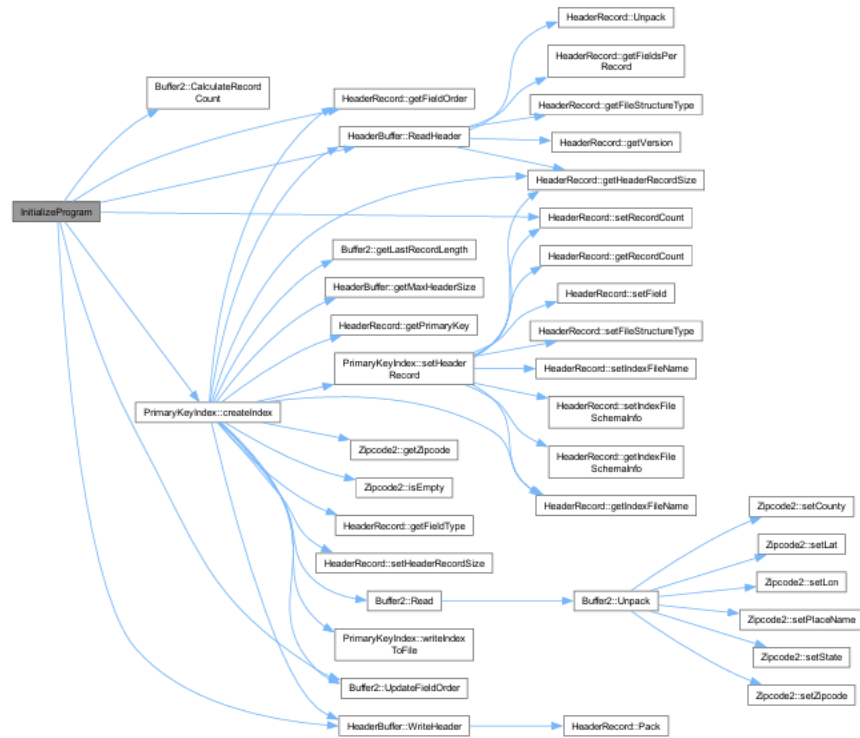
- csvFileName (to store the name of the CSV file)
- zcrFile (an object of fstream class for handling the ZCR file)
- recordCount (an integer to hold the number of records).

Functions:

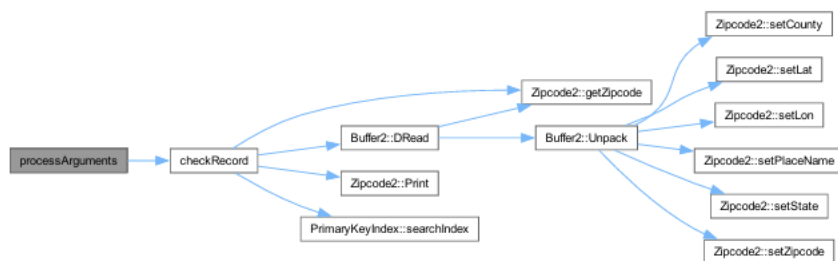
- main():
 - The main function of the program. It asks the user for the input file name, creates a Buffer2 object for the ZCR file, creates maps to hold the extreme coordinates for each state, processes the data, and prints the data.



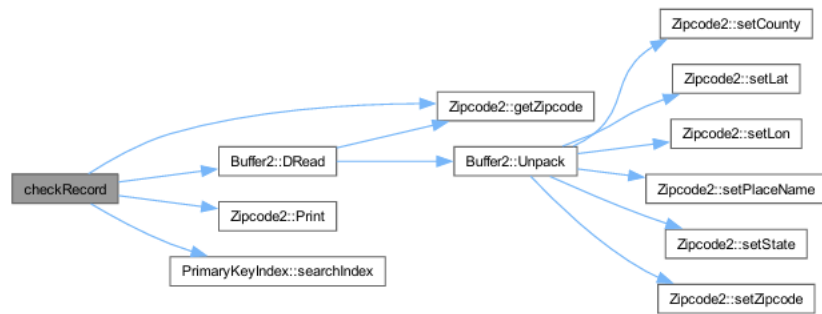
- InitializeProgram(std::fstream& dataFile, HeaderBuffer& hb, Buffer2& buffer, PrimaryKeyIndex& index, HeaderRecord& header):
 - A function to initialize the program by reading the header, updating the field order, calculating the record count, writing back the header, and creating the index file.



- processArguments(int argc, char* argv[], std::fstream& dataFile, PrimaryKeyIndex& index, Buffer2& buffer):
 - A function to process the command line arguments or user input to search for a record.

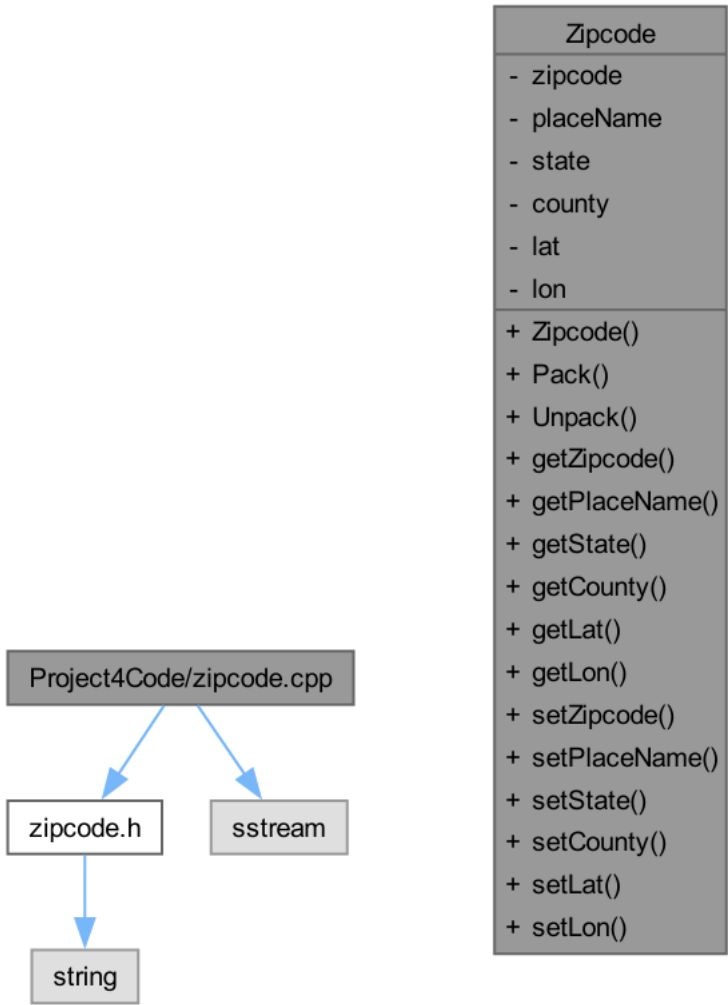


- `int checkRecord(std::fstream& dataFile, PrimaryKeyIndex& index, Buffer2& buffer, int zipcode):`
 - Use the zipcode (key) to search the index for offset
 - Use key/offset to Dread the file to access the record
 - Throw errors if the record does not exist
 - If the record is present, print



CLASSES

Class Name: Zipcode



Summary: The original iteration of the Zipcode object class. The Zipcode class is a fundamental building block in a system designed to process large datasets. It represents a zipcode record with associated geographic information.

Files: zipcode.h, zipcode.cpp

Subprograms: proj3.cpp, fileConverter.cpp

Usage: The Zipcode class provides methods to pack and unpack the data into a string for storage, making it easier to handle large amounts of data efficiently. It also provides getter and setter methods for each data field, allowing for easy access and modification of the data.

Data Structures/Types:

- int
- std::string
- double

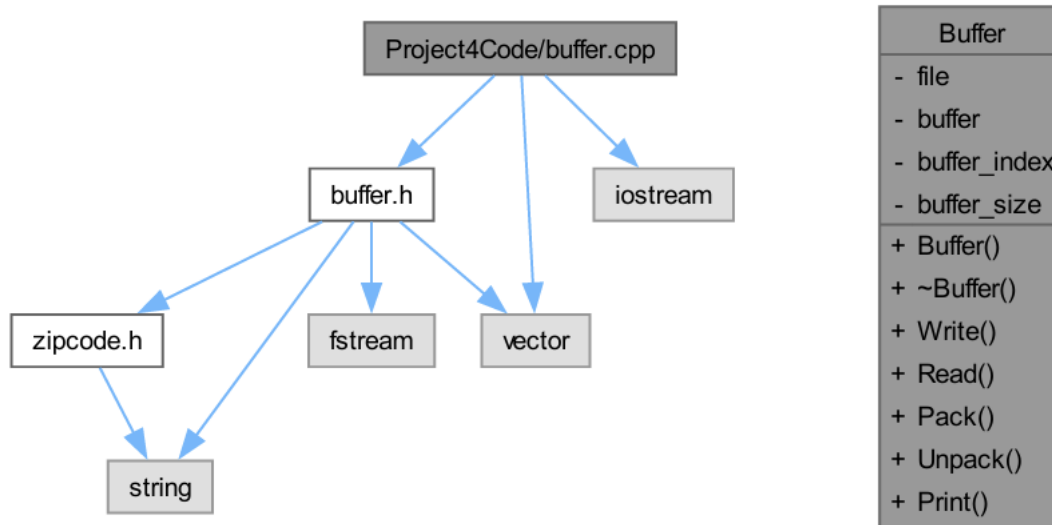
Variables:

- zipcode: type int storing the zipcode value
- placename: type string storing the place name
- state: type string containing the state code
- county: type string storing the count
- lat: type double storing the latitude
- lon: type double storing the longitude

Functions: The Zipcode class provides a default constructor, a Pack function to convert the object's data into a string for storage, an Unpack function to populate the object's data from a stored string, and getter and setter methods for each data field.

- Zipcode()
 - Zeros out each field to ensure proper "empty" status
- string Pack()
 - using stringstream to process the member variables into a single string to be returned
 - commas are placed in-between each field to maintain CSV formatting
- void Unpack(string)
 - using stringstream it takes a string to parse using a "," delimiter
 - each field is streamed into its corresponding data member, non-strings being converted to proper type

Class Name: Buffer



Summary: The Buffer class is a key component in a system designed to process large datasets, specifically zipcode records. It is responsible for reading and writing Zipcode objects to a file.

Files: buffer.h, buffer.cpp

Subprograms: proj3.cpp, fileConverter.cpp

Usage: The Buffer class uses a buffer to store data read from the CSV file and provides methods to pack and unpack Zipcode objects into strings that can be written to or read from the file. The buffer size is configurable, and the class provides error handling for file operations.

Data Structures/Types:

- `std::fstream` for file input/output operations
- `std::vector<std::string>` as a buffer to hold the data
- `size_t` for the current index in the buffer and the size of the buffer.
- `std::string` for the name of the file to open
- Class Zipcode for data processing

Variables:

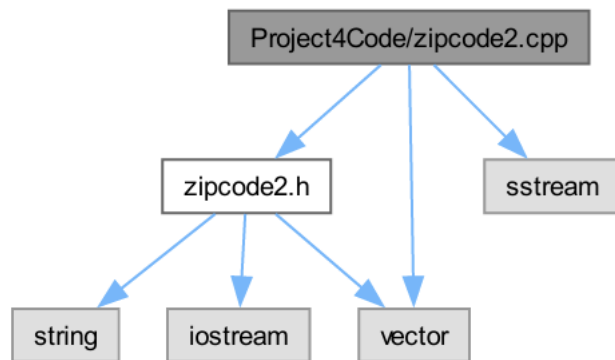
- `file`: a string containing the name of the file to read/write
- `buffer`: a vector of type string to help process data in memory and limit file accessing
- `buffer_index`: a `size_t` used for tracking current buffer position and help determine refill
- `buffer_size`: a `size_t` used to determine how big the buffer is with each fill (currently 2000)

Functions: The Buffer class provides a constructor that opens a file and fills the buffer initially, a destructor that closes the file if it is open, a Write function to write a Zipcode object to the file, a Read function to read a Zipcode object from the file, a Pack function to pack a Zipcode object into a

string, an Unpack function to unpack a string into a Zipcode object, and a Print function to print a Zipcode object.

- Buffer()
 - Default Constructor with initialization of buffer_index to 0
 - Opens up the data file for processing
 - Skip the CSV file header and fill buffer
- ~Buffer()
 - Destructor
 - Close the open file if the object is destroyed and the file is still open
- void Write(Zipcode&)
 - Takes a Zipcode object parameter to process
 - The stream is checked whether it is open or not
 - Uses helper function Pack(Zipcode) to convert to string
 - The string is passed to the file
- Zipcode Read()
 - Checks if buffer is empty, if so, refill
 - If buffer is empty after refill, return empty object
 - Use helper function Unpack(string) to process current buffer element
 - increment buffer
 - return zipcode
- string Pack(Zipcode&)
 - calls Zipcode Pack function to return the string record
- Zipcode Unpack(string)
 - Creates a tmp Zipcode object
 - Uses the Zipcode's unpack object
 - Returns the Zipcode object
- Void Print(Zipcode&)
 - Takes a Zipcode object to print
 - Goes sequentially and prints each element with a corresponding label to the console

Class Name: Zipcode2



Zipcode2
- zipcode
- placeName
- state
- county
- lat
- lon
+ Zipcode2()
+ getZipcode()
+ getPlaceName()
+ getState()
+ getCounty()
+ getLat()
+ getLon()
+ setZipcode()
+ setPlaceName()
+ setState()
and 7 more...

Summary: The Zipcode2 class is an updated version of the Zipcode class, used in Project 4. It represents a zipcode record with associated geographic information.

Files: zipcode2.h, zipcode2.cpp

Subprograms: project3Updated.cpp, findZipcode.cpp

Usage: The Zipcode2 class provides methods to pack and unpack the data into a string for storage, allowing for efficient handling of large amounts of data. It also provides getter and setter methods for each data field, allowing for easy access and modification of the data. An additional feature in this updated class is the ability to pack the data fields in a specified order.

Data Structures/Types:

- `std::vector<std::string>`: a parameter of pack method to specify the order of fields
- `int`
- `std::string`
- `double`

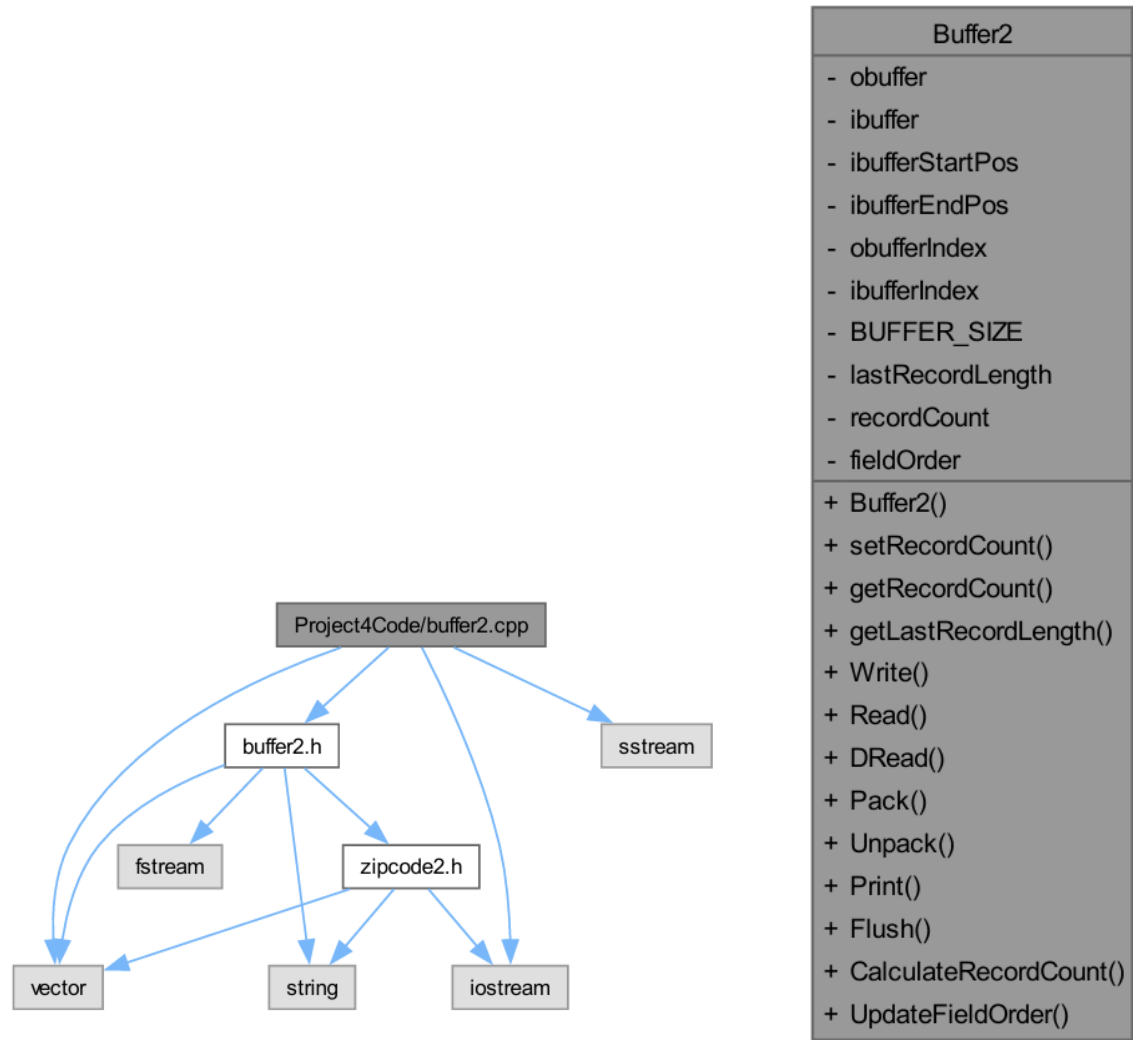
Variables:

- zipcode: type int storing the zipcode value
- placename: type string storing the place name
- state: type string containing the state code
- county: type string storing the count
- lat: type double storing the latitude
- lon: type double storing the longitude

Functions: The Zipcode2 class provides a default constructor, a Pack function to convert the object's data into a string for storage in a specified order, an Unpack function to populate the object's data from a stored string, getter and setter methods for each data field, a Print function to print the object's data, and an isEmpty function to check if the object is empty.

- Zipcode2()
 - Zeros out each field to ensure proper "empty" status
- string Pack(vector<string>&)
 - takes a parameter to determine field order for processing
 - using stringstream to process the member variables
 - the field is checked to determine write order into a single string to be returned
 - commas are placed in-between each field to maintain CSV formatting
 - remove the final trailing comma
- void Unpack(string)
 - using stringstream it takes a string to parse using a "," delimiter
 - each field is streamed into its corresponding data member, non-strings being converted to proper type
- Void Print()
 - Takes a Zipcode object to print
 - Goes sequentially and prints each element with a corresponding label to the console
- bool isEmpty()
 - if the zipcode matches the default, it is empty

Class Name: Buffer2



Summary: The Buffer2 class is an updated version of the Buffer class, used in Project 4. It is responsible for reading and writing Zipcode2 objects to a file.

Files: buffer2.h, buffer2.cpp

Subprograms: project3Updated.cpp, findZipcode.cpp

Usage: The Buffer2 class uses a buffer to store data read from the ZCR file and provides methods to pack and unpack Zipcode2 objects into strings that can be written to or read from the file. The buffer size is configurable, and the class provides error handling for file operations. An additional feature in this updated class is the ability to pack the data fields in a specified order.

Data Structures/Types:

- `std::fstream` for file input/output operations
- `std::vector<std::string>` as a buffer to hold the data
- `size_t` for the current index in the buffer and the size of the buffer.
- Class `Zipcode2` for data processing

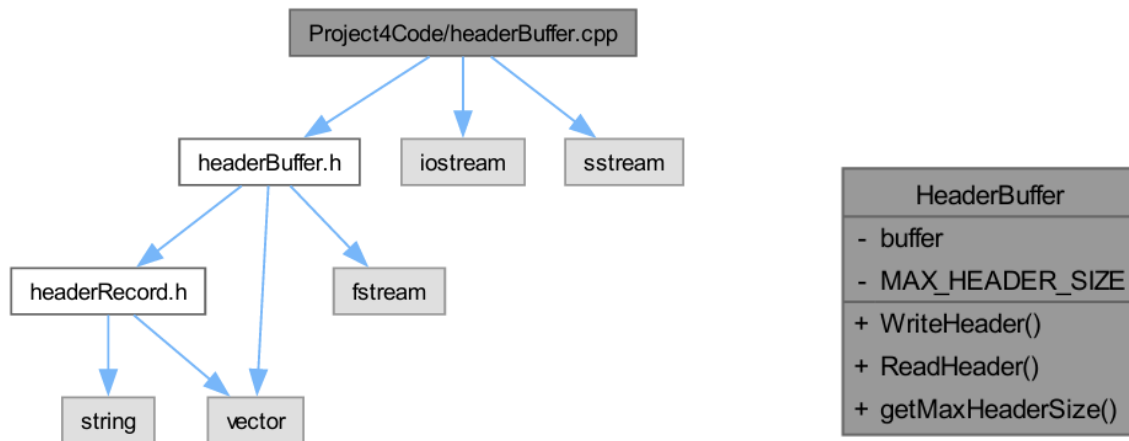
Variables:

- `obuffer`: type `vector<string>` used for writing to file, separate from `ibuffer` for flexibility
- `ibuffer`: type `vector<string>` used for reading from file, separate from `obuffer` for flexibility
- `obufferIndex`: tracks current position of `obuffer`
- `ibufferIndex`: tracks current position of `ibuffer`
- `BUFFER_SIZE`: type `const int` used to determine how large the buffer is to be filled with each read/write
- `lastRecordLength`: type `int` holding the value of the record length.
- `recordCount`: type `int` the number of records in the data file
- `fieldOrder`: `vector<string>` holding the field names in order of read/write

Functions: The `Buffer2` class provides a constructor that initializes the indices for the input and output buffers to 0, a `Write` function to write a `Zipcode2` object to the file, a `Read` function to read a `Zipcode2` object from the file, a `DRead` function to read a `Zipcode2` object from a file using a key, a `Pack` function to pack a `Zipcode2` object into a string, an `Unpack` function to unpack a string into a `Zipcode2` object, a `Print` function to print a `Zipcode2` object, a `Flush` function to flush the output buffer to a file, a `CalculateRecordCount` function to calculate the record count in a file, and an `UpdateFieldOrder` function to update the field order for packing/unpacking.

- `Buffer()`
 - Default Constructor with initialization of `ibufferIndex` and `obufferIndex` to 0
 - Initializes field order to standard field order
- `Int Write(Zipcode2&, fstream&)`
 - Check if `fstrean` is open, return -1 if not
 - Use helper function `Pack()` to pack zipcode
 - If first value is not a digit, return -1 and exit
 - Add to buffer
 - If buffer is filled, push string to file
- `Zipcode2 Read(fstream&, int)`
 - Check if file is open, if not, return
 - If buffer is empty, refill
 - If pass buffer fill and still empty, return empty `Zipcode2`
 - Process buffer to isolate record from length and unpack
 - `lastRecordLength` is used to constrain record to appropriate limits on top of the delimiter
 - try to process string, catch error
- `Zipcode2 DRead(fstream&, int, int)`
 - Takes a `fstream`, key (zipcode), and offset

- Clear file flags and move to file position based on offset
 - Ensure file is open
 - Process record read with Unpack
- string Pack(Zipcode2&)
 - calls zipcode.Pack() and passes the current field order
- Zipcode2 Unpack(string)
 - Uses stringstream to parse a string
 - Matches fields and sets values to temporary Zipcode2 object
 - Returns Zipcode2 object
- Void Print(Zipcode&)
 - If zipcode is not empty, call zipcode2 print
- Void Flush(&fstream)
 - Check if file is open
 - Write all remaining to outfile
 - Flush outfile
- Int CalculateRecordCount(fstream&)
 - Clear file flags, save curr Position, and position at top of file
 - Iterate through the file, Continue loop if non-records
 - Increment counter if passed non-record check
 - Return to curr position
- Void UpdateFieldOrder(vector<string>)
 - Copy passed field order with current

Class Name: HeaderBuffer

Summary: The HeaderBuffer class is responsible for reading the header from a file. It clears the buffer, reads the header from the file, unpacks the header into the HeaderRecord object, and checks for consistency.

Files: headerBuffer.h, headerBuffer.cpp

Subprograms: fileConverter.cpp, project3Updated.cpp, findZipcode.cpp

Usage: The HeaderBuffer class is used to read the header from a file. It uses a buffer to store the data read from the file and provides methods to unpack the header into a HeaderRecord object and check for consistency. The class provides error handling for file operations.

Data Structures/Types:

- std::fstream for file input/output operations
- std::vector<std::string> as a buffer to hold the data
- HeaderRecord for header processing
- int

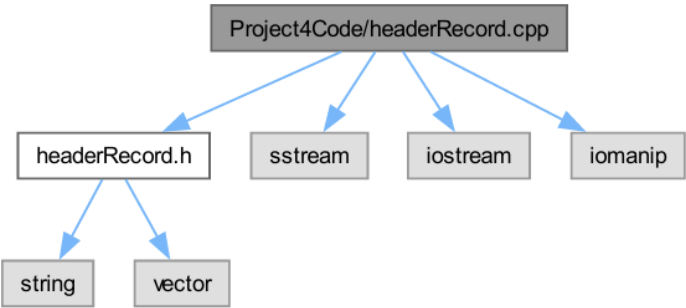
Variables:

- buffer: type vector<string> used for reading from file
- MAX_HEADER_SIZE: type int user to set fixed header size

Functions: The HeaderBuffer class provides a ReadHeader function that reads the header from a file, unpacks the header into the HeaderRecord object, and checks for consistency.

- Int WriteHeader(HeaderRecord&, fstream&)
 - Try-catch, header record Pack()
 - Calculate padding to meet fixed size
 - If padding <0, too big and throw error
 - Add padding and end with \n
 - Write to file
- Bool ReadHeader(HeaderRecord&, fstream&)
 - Clears the buffer and reads the header from the file
 - Unpacks the header into the HeaderRecord object
 - Checks for consistency of the header
 - Returns true if the header was read successfully and is consistent, false otherwise
 - Handles exceptions during the process

Class Name: HeaderRecord



HeaderRecord
- fileStructureType
- version
- headerRecordSize
- recordSizeInteger
- sizeFormatType
- indexFileName
- indexFileSchemaInfo
- recordCount
- fieldsPerRecord
- fields
- primaryKey
+ HeaderRecord()
+ getFileStructureType()
+ getVersion()
+ getHeaderRecordSize()
+ getRecordSizeInteger()
+ getSizeFormatType()
+ getIndexFileName()
+ getIndexFileSchemaInfo()
+ getRecordCount()
+ getFieldsPerRecord()
and 17 more...

Summary: The HeaderRecord class represents a header record in a file. It provides methods to pack and unpack the data into a string for storage, as well as getter and setter methods for each data field.

Usage: The HeaderRecord class is used to contain and process header information for a ZRC and IDX file. It provides methods to set a field in the record, pack the header record into a string, unpack a string into a header record, print the header record, and get the order of the fields in the record.

Data Structures:

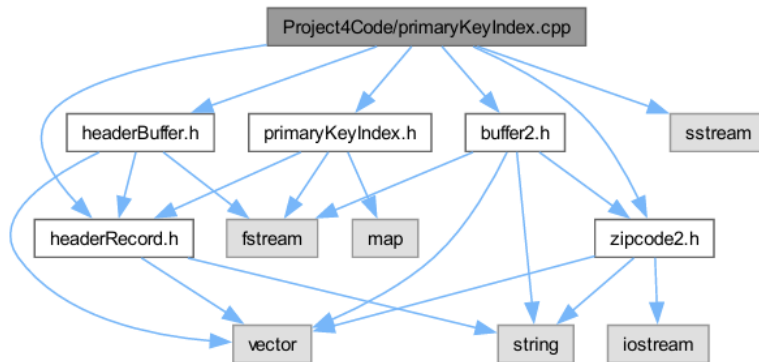
- std::string for string manipulation
- std::vector<std::pair<std::string, std::string>> for dynamic array operations
- int

Variables:

- fileStructureType: type string used for the file structure type
- version: type string used for the version
- headerRecordSize: type int used for the header record size
- recordSizeInteger: type int used for the record size as an integer
- sizeFormatType: type string used for the size format type
- indexFileName: type string used for the index file name
- indexFileSchemaInfo: type string used for the index file schema information
- recordCount: type int used for the record count
- fieldsPerRecord: type int used for the number of fields per record
- fields: type vector<pair<string, string>> used for the fields
- primaryKey: type string used for the primary key

Functions: The HeaderRecord class provides a constructor, a setField function to set a field in the record, a Pack function to pack the header record into a string, an Unpack function to unpack a string into a header record, a Print function to print the header record, and a getFieldOrder function to get the order of the fields in the record.

- HeaderRecord(int numFields = 6)
 - Default constructor for HeaderRecord
 - If 6 is passed, then the fields will populate with zipcode
 - If not, they will be initialized “”,””
- void setField(int index, const std::string& fieldName, const std::string& fieldType)
 - Sets a field in the record by index
- std::string getFieldType(const std::string& fieldName) const
 - use fieldName and return value
- std::string Pack()
 - Use ostream to pack sequentially
 - Label provide with each value
 - Fields loop to fill
 - ;\n placed at the end to delimitate and distinguish from record
- void Unpack(std::string str)
 - Process string
 - : used to delimitate label from value
 - Skip field, read value until ;
 - Fields have an additional , separation
- void Print()
 - Prints the header record sequentially with labels
- std::vectorstd::string getFieldOrder() const
 - Gets the order of the fields in the record

Class Name: PrimaryKeyIndex

PrimaryKeyIndex
- indexFile
- intIndex
- headerOffset
+ PrimaryKeyIndex()
+ ~PrimaryKeyIndex()
+ createIndex()
+ writeIndexToFile()
+ readIndexFromFile()
+ searchIndex()
+ setHeaderRecord()

Summary: The PrimaryKeyIndex class is an index based on the primary key of the zipcode records. It is used to handle the primary key index for better file access in large datasets.

Files: primaryKeyIndex.h, primaryKeyIndex.cpp

Subprograms: findZipcode.cpp

Usage: The PrimaryKeyIndex class is used to create, read, and write an index from a file. It also provides methods to search the index for a zipcode and set the HeaderRecord objects for the data and index files.

Data Structures:

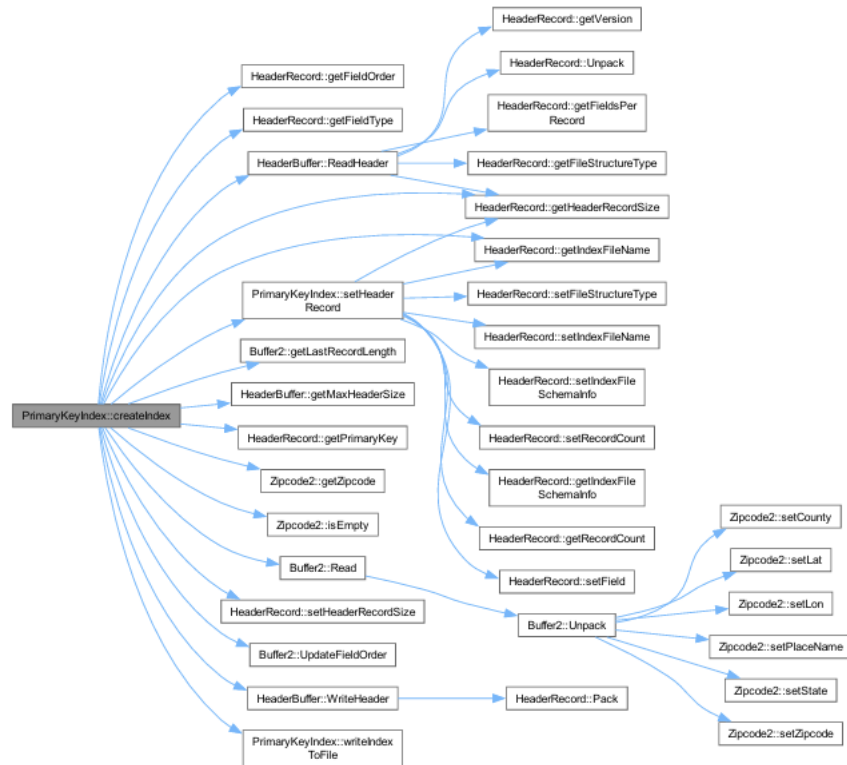
- std::fstream for file input/output operations
- std::map<int, int> for storing the primary key index
- int
- HeaderRecord objects for data header information and index header information
- HeaderBuffer for processing data header information and index header information
- Buffer2 for reading from the file to provide records and offsets
- Zipcode2 for temporary storage for key generation

Variables:

- indexFile: type fstream used for the index file stream
- intIndex: type map<int, int> used for the primary key index
- headerOffset: type int used to skip the header when reading the records

Functions: The PrimaryKeyIndex class provides a constructor and destructor, a createIndex function to create an index from a file, a writeIndexToFile function to write the index to a file, a readIndexFromFile function to read the index from a file, a searchIndex function to search the index for a zipcode, and a setHeaderRecord function to set the HeaderRecord objects for the data and index files.

- ~PrimaryKeyIndex()
 - If file is not closed when object is destroyed, close file
- bool createIndex(std::fstream& inFile)
 - Check if file is open, close if it is not
 - Create HeaderRecord, HeaderBuffer, and Buffer2 object to process ZCR file
 - Read header information into HeaderRecord object
 - Update field order provided by ZCR file into Buffer2 for proper usage
 - Use the dataHeader HeaderRecord to set indexFileName. Open or Create IDX file
 - Validate that it has opened
 - Get Primary with associated field
 - Set header offset
 - Using temporary Zipcode2 and Buffer2, generate key, offset pair for the index map
 - Offset is cumulative
 - Generate header for IDX file
 - 2 fields (instead of 6)
 - Key, Offset
 - Use helped function setHeaderRecord to set values
 - Write header to indexfile
 - Write index to file
 - Clear index and close file if it is still open



- void setHeaderRecord(HeaderRecord& dataHR, HeaderRecord& indexHR)
 - Sets File Structure Type to IDX for index
 - Set headerOffset from data file
 - Set index file name from data file
 - Set index schema from data file
 - Set record count from data file
 - Set fields with predetermined Key, Offset
- void writeIndexToFile(std::fstream& outFile)
 - Cycle through index map and write to index file until empty
- void readIndexFromFile(std::fstream& inFile)
 - Check if open
 - Position with headerOffset
 - Parse string from idx file
 - If value matches key/offset, validate and add to index map
 - Else, error reading
- int searchIndex(int zipcode)
 - Searches the index map for a zipcode

NEXT STEPS

The current iteration of the system has successfully achieved its goal of efficiently analyzing geographical data from a custom ZCR file and displaying extreme zip codes for each state. It has also provided functionality to convert a CSV file into a custom ZCR file, including a header and length indicated records. An index file (IDX) is generated and utilized to quickly locate specific records by zip code. The record is located based on user input from the command line with the -Z flag. The record is displayed if found, or a message is displayed stating the record is not present. Multiple records can be accessed at one execution of the program.

The system's design and implementation have been thoroughly documented in this design document, providing a comprehensive guide for understanding the system's design and functionality. The document serves as a valuable resource for software developers, testers, and maintainers who are involved in the development, testing, and maintenance of the system. It can also be used by end-users who are interested in understanding the system's design and functionality.

Looking forward to the next iteration, there are several potential enhancements and improvements that could be made to the system. These include:

- **Optimization:** There are always opportunities to further optimize the system for performance and efficiency. The buffer system should be refined to operate more efficiently with. Classes may need to be refactored to reduce coupling and emphasize OOP principles. In the next iteration, we could conduct a thorough performance analysis of the system and implement optimizations where necessary.
- **Enhanced Search Functionality:** The current system allows users to search for a specific record by zipcode. In the next iteration, we could enhance the search functionality to allow users to search for records based on other criteria, such as city or state.
- **Additional Data Processing Features:** The current system provides basic data processing features, such as finding extreme zip codes for each state. In the next iteration, we could add more advanced data processing features, such as calculating the average latitude and longitude for each state, or finding the most populous city in each state.
- **Improved User Interface:** While the current system is designed to be executed from the command line, we could develop a more interactive command-line interface (CLI) in the next iteration to make the system more user-friendly and intuitive to use. This could involve implementing a menu system that guides the user through the various functionalities of the program.

In conclusion, while the current iteration of the system has been successful, there are many exciting possibilities for future development and improvement. We look forward to continuing to evolve and enhance the system in the next iteration. We appreciate the continued support and feedback from our users, which is invaluable in helping us to improve and refine the system. Thank you for your interest in our system and for taking the time to read this design document. We hope you find it informative and helpful.